

FIREBASE SCHEMA FOR WEB APP

This document defines a teammate-friendly Firebase Realtime Database schema for the SmartGym web app.

The goal is simple:

- easy to understand in Firebase Console
- easy to explain to a frontend teammate
- still rich enough for session analysis
- preserves rep-level detail

Product Goal

The web app should help staff, coaches, and athletes:

- see which user trained on which day
- see what exercises they did
- track weekly and daily progress
- inspect machine-specific trends
- open one session and analyze sets and reps in detail

Design Decision

We keep sessions inside the week tree.

This is the best tradeoff for the current stage of the project because:

- it is easier to understand quickly
- all workouts for a user live under the correct week/day
- your teammate does not have to jump between summary and detail roots
- rep-level detail can still be preserved

The important part is not splitting data into many roots. The important part is organizing each session clearly.

Recommended Top-Level Nodes

Use these roots:

- 'usersByRfid/{uid}'
- 'calibrations/{uid}/{machineTypeId}'
- 'calibrationHistory/{uid}/{machineTypeId}/{epoch}' optional
- 'machineConfigs/{machineId}'

- 'devices/{deviceId}'
- 'bodyMetricsHistory/{uid}/{dateKey}' web-app owned
- 'sessionFeedback/{uid}/{sessionId}' web-app owned
- 'athleteWeeklySessions/{uid}/{weekKey}'

Main Rule

Everything about a user's workout week lives under:

```
""json
athleteWeeklySessions/{uid}/{weekKey}
""
```

Inside that week:

- 'weekSummary' is for weekly overview
- 'days/{dayKey}/daySummary' is for daily overview
- 'days/{dayKey}/timeline' is for chronological session cards
- 'days/{dayKey}/sessions/{sessionId}' is the full session record

This keeps one intuitive location for the entire workout history.

Top-Level Nodes

1. Users

Path:

```
""json
usersByRfid/{uid}
""
```

Purpose:

- athlete identity
- body stats
- current goal

Example:

```
""json
{
  "rfidUid": "D6-FA-A5-05",
  "displayName": "Usuario1",
  "hasBasicData": true,
  "weightKg": 78.0,
```

```
"age": 22,  
"heightCm": 175.0,  
"goal": "hypertrophy",  
"updatedAtEpoch": 1776691800,  
"updatedAtIso": "2026-04-20T08:30:00-0500"  
}
```

2. Current Calibrations

Path:

```
““json  
calibrations/{uid}/{machineTypeId}  
““
```

Purpose:

- current effective calibration
- current suggested load
- current user ROM threshold

Example:

```
““json  
{  
  "machineTypeId": "leg_ext",  
  "hasCalibration": true,  
  "suggestedWeightKg": 22.0,  
  "userRomPercent": 91.0,  
  "updatedAtEpoch": 1776691800,  
  "updatedAtIso": "2026-04-20T08:30:00-0500"  
}  
““
```

3. Calibration History

Path:

```
““json  
calibrationHistory/{uid}/{machineTypeId}/{epoch}  
““
```

Purpose:

- historical trend
- auditing changes
- calibration evolution charts

This is optional for the first web app version. If the tree feels too cluttered, this is the first node that can be postponed.

4. Machine Configs

Path:

```
""json
machineConfigs/{machineId}
""
```

Purpose:

- machine metadata
- ROM targets
- recommendation presets
- encoder calibration values for wire/linear motion sensors

Frontend can use this for:

- machine labels
- admin/config screens
- thresholds for progress views
- calibration tools for draw-wire encoders
- exercise grouping and muscle-group dashboards

Recommended taxonomy fields:

```
""json
{
  "exerciseCategory": "strength_machine",
  "primaryMuscleGroup": "quadriceps",
  "secondaryMuscleGroup": "knees"
}
""
```

Recommended encoder fields:

```
""json
{
  "encoderCalibrationValid": true,
  "encoderZeroRaw": 1240,
  "encoderFullRaw": 2875,
  "encoderReferenceDistanceMm": 1000.0,
  "encoderInvertDirection": false
}
""
```

Recommended calibration flow:

- move the carriage to the zero point
- capture the raw reading as 'encoderZeroRaw'
- move the carriage a known distance, usually '1000 mm'
- capture the raw reading as 'encoderFullRaw'
- save the actual distance in 'encoderReferenceDistanceMm'
- set 'encoderInvertDirection' if the reading moves opposite to the motion

5. Devices

Path:

```
""json
devices/{deviceId}
""
```

Purpose:

- machine display status
- online/offline info
- active user / active machine state

Useful for:

- operations dashboard
- gym floor monitoring
- live machine occupancy
- device-health checks

6. Body Metric History

Path:

```
""json
bodyMetricsHistory/{uid}/{dateKey}
""
```

Purpose:

- bodyweight trend charts
- transformation dashboards
- weekly compliance and check-in flows

Example:

```
""json
```

```
{
  "dateKey": "2026-04-20",
  "weightKg": 77.4,
  "bodyFatPercent": 15.8,
  "skeletalMusclePercent": 41.2,
  "waistCm": 81.0,
  "sleepHours": 7.4,
  "energyScore": 4,
  "notes": "Felt fresh and recovered",
  "updatedAtEpoch": 1776691800,
  "updatedAtIso": "2026-04-20T08:30:00-0500"
}
```

7. Session Feedback

Path:

```
““json
sessionFeedback/{uid}/{sessionId}
““
```

Purpose:

- athlete-entered RPE
- pain / soreness flags
- post-session notes
- dashboard overlays without changing device firmware

Example:

```
““json
{
  "sessionId": "session_1776146400_1",
  "rpe": 8,
  "energyScore": 4,
  "sorenessScore": 2,
  "painScore": 0,
  "readinessScore": 4,
  "notes": "Strong set quality, last set slowed down",
  "updatedAtEpoch": 1776147700,
  "updatedAtIso": "2026-04-13T07:21:40-0500"
}
““
```

Weekly Tree

Path:

```
""json
athleteWeeklySessions/{uid}/{weekKey}
""
```

Purpose:

- weekly browsing
- daily browsing
- session detail
- progress analytics

Recommended shape:

```
""json
{
  "weekSummary": {},
  "days": {
    "2026-04-13": {
      "meta": {},
      "daySummary": {},
      "timeline": {},
      "sessions": {}
    }
  }
}
""
```

Week Summary

Path:

```
""json
athleteWeeklySessions/{uid}/{weekKey}/weekSummary
""
```

Purpose:

- fast weekly dashboard
- weekly charts
- counts and totals

Recommended fields:

```
""json
{
```

```

"scope": "week",
"lastSessionId": "session_1776146400_1",
"lastStartedAtEpoch": 1776146400,
"lastStartedAtIso": "2026-04-13T07:00:00-0500",
"totalSessions": 7,
"totalValidReps": 238,
"totalInvalidReps": 9,
"totalSetsCompleted": 26,
"totalDurationMs": 4235000,
"totalRestMs": 811000,
"totalFastEccentricWarnings": 12,
"totalVolumeLoadKg": 6128.5,
"avgRomPercent": 86.4,
"avgPeakVelocityPctPerSec": 62.8,
"machineTypeCounts": {
  "leg_ext": 3,
  "lat_pull": 2,
  "chest_press": 2
},
"goalCounts": {
  "hypertrophy": 7
}
}

```

Day Node

Path:

```

""json
athleteWeeklySessions/{uid}/{weekKey}/days/{dayKey}
""

```

Inside a day:

- 'meta'
 - latest session metadata for that day
- 'daySummary'
 - aggregate day metrics
- 'timeline'
 - chronological session cards
- 'sessions'
 - full sessions for that day

This makes it easy to answer:

- what did this user do today?

- in what order?
- how many sets / reps?
- how well did they perform?

Timeline

Path:

```
""json
athleteWeeklySessions/{uid}/{weekKey}/days/{dayKey}/timeline/{timelineKey}
""
```

Purpose:

- lightweight session card list
- chronological rendering

Recommended fields:

```
""json
{
  "sessionId": "session_1776146400_1",
  "dayKey": "2026-04-13",
  "weekKey": "2026-W16",
  "identity": {
    "userId": "D6-FA-A5-05",
    "userDisplayName": "Usuario1"
  },
  "ordering": {
    "startedAtEpoch": 1776146400,
    "endedAtEpoch": 1776147668,
    "startedAtIso": "2026-04-13T07:00:00-0500",
    "endedAtIso": "2026-04-13T07:21:08-0500"
  },
  "machine": {
    "machineId": "leg_ext_1",
    "machineTypeId": "leg_ext",
    "machineDisplayName": "Leg Extension",
    "exerciseCategory": "strength_machine",
    "primaryMuscleGroup": "quadriceps",
    "secondaryMuscleGroup": "knees"
  },
  "plan": {
    "goal": "hypertrophy",
    "selectedWeightKg": 23.0,
    "targetSets": 4,
    "targetRepsMin": 10,
```

```

    "targetRepsMax": 15
  },
  "summary": {
    "setsCompleted": 4,
    "validReps": 46,
    "invalidReps": 1,
    "avgRomPercent": 88.3,
    "durationMs": 1268000
  }
}

```

Sessions

Path:

```

""json
athleteWeeklySessions/{uid}/{weekKey}/days/{dayKey}/sessions/{sessionId}
""

```

This is the session record your teammate should open when they need detailed analysis.

Recommended internal structure:

- 'identity'
- 'machine'
- 'timing'
- 'plan'
- 'summary'
- 'analysis'
- 'setOverview'
- 'setDetails'
- 'repSets'

This keeps one intuitive path while still separating the session into logical subsections.

Recommended Session Shape

```

""json
{
  "sessionId": "session_1776146400_1",
  "identity": {},
  "machine": {},
  "timing": {},
  "plan": {},

```

```

"summary": {},
"analysis": {},
"setOverview": [],
"paths": {
  "setDetails": "setDetails",
  "repSets": "repSets"
},
"representativeReps": {},
"setDetails": {
  "sessionId": "session_1776146400_1",
  "setCount": 4,
  "sets": {}
},
"repSets": {
  "set1": {},
  "set2": {},
  "set3": {},
  "set4": {}
}
}
""

```

What Each Session Section Is For

‘identity’

Who performed the session.

Example:

```

""json
{
  "userId": "D6-FA-A5-05",
  "userDisplayName": "Usuario1",
  "anonymous": false
}
""

```

‘machine’

Which machine was used.

Example:

```

""json
{
  "machineId": "leg_ext_1",

```

```
"machineTypeId": "leg_ext",
"machineDisplayName": "Leg Extension",
"exerciseCategory": "strength_machine",
"primaryMuscleGroup": "quadriceps",
"secondaryMuscleGroup": "knees",
"machineIdealRomPercent": 90.0
}
```

‘timing‘

When the session happened.

Example:

```
“json
{
  "startedAtEpoch": 1776146400,
  "endedAtEpoch": 1776147668,
  "startedAtIso": "2026-04-13T07:00:00-0500",
  "endedAtIso": "2026-04-13T07:21:08-0500",
  "startMs": 1776146400000,
  "endMs": 1776147668000,
  "durationMs": 1268000,
  "totalRestMs": 188000
}
```

‘plan‘

What the workout intended to do.

Example:

```
“json
{
  "goal": "hypertrophy",
  "calibrationBased": true,
  "selectedWeightKg": 23.0,
  "suggestedWeightKg": 22.0,
  "userRomPercent": 91.0,
  "targetSets": 4,
  "targetRepsMin": 10,
  "targetRepsMax": 15,
  "plannedRestSeconds": 60
}
```

'summary'

Fast high-level results.

Example:

```
““json
{
  "setsCompleted": 4,
  "setCount": 4,
  "repCount": 47,
  "validReps": 46,
  "invalidReps": 1,
  "fastEccentricWarnings": 2,
  "validRepRate": 97.87,
  "volumeLoadKg": 1058.0,
  "avgRomPercent": 88.3,
  "bestRomPercent": 93.5,
  "avgConcentricTimeMs": 1018.4,
  "avgPeakVelocityPctPerSec": 64.7,
  "avgPeakEccentricVelocityPctPerSec": 71.2
}
““
```

'analysis'

Derived metrics for progress tracking.

This is where your richer coaching logic belongs.

Recommended metrics:

- 'firstSetAvgRomPercent'
- 'lastSetAvgRomPercent'
- 'firstSetAvgPeakVelocityPctPerSec'
- 'lastSetAvgPeakVelocityPctPerSec'
- 'validAtOrAboveUserRomCount'
- 'validBelowUserRomCount'
- 'validAtOrAboveIdealRomCount'
- 'romComplianceRate'
- 'idealRomHitRate'
- 'avgRestSecondsPerSet'
- 'fatigueRomDrop'
- 'fatigueVelocityDrop'
- invalid reason counters

'setOverview'

Quick per-set table.

Useful for:

- set comparison cards
- fatigue progression chart
- simple coach review

'setDetails'

More explicit set-by-set detail.

Useful when the frontend wants:

- dedicated set pages
- richer tables
- breakdown by set without scanning all reps

'repSets'

This is where rep-level detail lives.

Purpose:

- preserve every rep
- support deep session analysis
- support future charts and coaching tools

Recommended shape:

```
““json
"repSets": {
  "set1": {
    "sessionId": "session_1776146400_1",
    "setNumber": 1,
    "count": 12,
    "setSummary": {},
    "reps": [
      {
        "index": 0,
        "setNumber": 1,
        "repNumberInSet": 1,
        "valid": true,
        "warningFastEccentric": false,
        "selectedWeightKg": 23.0,
        "romPercent": 91.4,
        "durationMs": 2580,
        "concentricTimeMs": 980,
```

```

    "peakVelocityPctPerSec": 67.1,
    "peakEccentricVelocityPctPerSec": 73.0,
    "invalidFlags": 0,
    "offsetMs": 0
  }
]
}
}
""

```

This is the section to keep if analysis quality matters.

What To Remove If Firebase Feels Cluttered

If you want to simplify without losing useful analysis, remove in this order:

1. 'calibrationHistory'
2. extra cosmetic fields in 'representativeReps'
3. analytics fields you are not showing in the web app yet

Do not remove these if analysis matters:

- 'summary'
- 'analysis'
- 'setOverview'
- 'setDetails'
- 'repSets'

What The Web App Should Read

Dashboard

Use:

- 'usersByRfid'
- 'athleteWeeklySessions/{uid}/{currentWeek}/weekSummary'
- 'devices'

User Progress Page

Use:

- 'usersByRfid/{uid}'
- 'calibrations/{uid}'
- 'bodyMetricsHistory/{uid}'
- 'sessionFeedback/{uid}'

- 'athleteWeeklySessions/{uid}'

Optionally:

- 'calibrationHistory/{uid}'

Weekly View

Use:

- 'athleteWeeklySessions/{uid}/{weekKey}/weekSummary'
- 'athleteWeeklySessions/{uid}/{weekKey}/days'

Day View

Use:

- 'athleteWeeklySessions/{uid}/{weekKey}/days/{dayKey}/daySummary'
- 'athleteWeeklySessions/{uid}/{weekKey}/days/{dayKey}/timeline'

This is enough to answer:

- what exercises did the user do today?
- at what times?
- how many sessions?

Session Detail View

Use:

- 'athleteWeeklySessions/{uid}/{weekKey}/days/{dayKey}/sessions/{sessionId}'

This gives the full analysis without needing another root.

Team Contract

If your teammate remembers one rule, it should be:

- profiles live in 'usersByRfid'
- current machine personalization lives in 'calibrations'
- weekly and daily browsing lives in 'athleteWeeklySessions'
- deep session analysis also lives inside that session node

In other words:

- one week tree for browsing
- one session node inside that tree for detail

Recommended Next Step

The best next practical move is:

1. keep the single-tree design
2. clean the session node into clear sections
3. update the sample Firebase seed to match this cleaner format
4. only remove optional nodes, not analysis detail

That gives you a schema that is easy to explain and still good enough for real session analysis.